arXiv:2307.02053v1 [cs.CL] 5 Jul 2023

FLACUNA: Unleashing the Problem Solving Power of VICUNA using FLAN Fine-Tuning

Deepanway Ghosal[‡], **Yew Ken Chia**[‡], **Navonil Majumder**[†], **Soujanya Poria**[‡]
[‡] DeCLaRe Lab, Singapore University of Technology and Design, Singapore
{deepanway_ghosal, yewken_chia}@mymail.sutd.edu.sg
{navonil_majumder,sporia}@sutd.edu.sg

CODE: https://github.com/declare-lab/flacuna
MODEL: https://huggingface.co/declare-lab/flacuna-13b-v1.0
FLAN-MINI: https://huggingface.co/declare-lab/flan-mini

Abstract

Recently, the release of INSTRUCTEVAL [Chia et al., 2023] has provided valuable insights into the performance of large language models (LLMs) that utilize encoder-decoder or decoder-only architecture. Interestingly, despite being introduced four years ago, T5-based LLMs, such as FLAN-T5, continue to outperform the latest decoder-based LLMs, such as LLAMA and VICUNA, on tasks that require general problem-solving skills. This performance discrepancy can be attributed to three key factors: (1) Pre-training data, (2) Backbone architecture, and (3) Instruction dataset. In this technical report, our main focus is on investigating the impact of the third factor by leveraging VICUNA, a large language model based on LLAMA, which has undergone fine-tuning on ChatGPT conversations. To achieve this objective, we fine-tuned VICUNA using a customized instruction dataset collection called FLAN-MINI. This collection includes a subset of the large-scale instruction dataset known as FLAN, as well as various code-related datasets and conversational datasets derived from ChatGPT/GPT-4. This dataset comprises a large number of tasks that

Preprint. Under review.

demand problem-solving skills. Our experimental findings strongly indicate that the enhanced problem-solving abilities of our model, FLACUNA, are obtained through fine-tuning VICUNA on the FLAN dataset, leading to significant improvements across numerous benchmark datasets in INSTRUCTEVAL. FLACUNA is publicly available at `https://huggingface.co/declare-lab/flacuna-13b-v1.0`.

1 Introduction

ChatGPT and its successor GPT-4 have surpassed their prior state-of-the-art models on a vast majority of the benchmarking tasks and datasets. However, to preserve privacy, natively running a 175B+ sized model like GPT-3 is beyond the capabilities of most organizations, let alone individuals. This has prompted many researchers to fine-tune manageable-sized LLMs — from 7B to 30B on a diverse set of instruction examples generated by ChatGPT or GPT-4. This has birthed LLMs, such as, Alpaca [Taori et al., 2023] and VICUNA [Chiang et al., 2023] that are fine-tuned checkpoints of LLaMA [Touvron et al., 2023]. These models have attained close to ChatGPT-level performance on some specific benchmarking tasks, but overall generalization still remains elusive. Recent works like INSTRUCTEVAL [Chia et al., 2023] strongly hint that the fine-tuning datasets dictate the task-specific performances. For instance, it has been observed that FLAN-T5 — a T5 checkpoint fine-tuned on FLAN Collection instruction dataset — outperforms VICUNA and Alpaca on tasks involving strong reasoning and problem-solving skills. This spurred us to fine-tune VICUNA on FLAN-MINI Collection dataset, anticipating improvement on reasoning-intensive tasks in INSTRUCTEVAL [Chia et al., 2023].

To this end, we first sample a 1M-sized instruction dataset from the 15M-sized FLAN Collection dataset [Longpre et al., 2023] and combined it with several other datasets comprising coding tasks and ChatGPT/GPT-4 distilled conversations. The resulting smaller dataset, FLAN-MINI, is then cast into the conversational format of VICUNA. To ensure a reasonable computational cost for the fine-tuning process, we retrofit LoRA [Hu et al., 2021] adapter into the LLaMA [Touvron et al., 2023] decoder-transformer of VICUNA. Following a parameter-efficient LoRA fine-tuning of the VICUNA checkpoint on FLAN-MINI, we obtain FLACUNA. As expected, FLACUNA outperforms VICUNA by a substantial margin on most benchmark datasets, especially for reasoning-intensive tasks. However, the performance of FLACUNA still remains below FLAN-T5 on the same reasoning benchmarks. This could be attributed to the 15-times smaller dataset of the instruction dataset which may contain less diverse samples. Furthermore, full fine-tuning of VICUNA may narrow the gap with FLAN-T5.

This work overall has the following contributions:

1. Improving the problem-solving capability of VICUNA through parameter efficient fine-tuning on FLAN-MINI.
2. Introducing an instruction tuning dataset, FLAN-MINI, comprising a diverse set of tasks and templates.

2 Training Details

Preparing the FLAN-MINI Collection. Given the enormous size of the FLAN Collection [Longpre et al., 2023], we opted to work with a carefully selected subset that maintains a high level of task diversity while reducing the overall dataset size. In Table 1, we present the specific tasks included in our subset of FLAN, along with their respective dataset sizes. As the public release of the FLAN Collection does not include programming tasks, we augment the collection with existing code datasets. Specifically, we include CodeContests [Li et al., 2022a], APPS [Hendrycks et al., 2021a] and CodeSearchNet [Husain et al., 2019a]. Following the data processing pipeline of FLAN Collection, we sample a fixed number of examples from each dataset, where each example is randomly augmented with different prompt templates. Specifically, the examples are processed with a pool of handcrafted prompt templates and may be used as zero-shot examples or grouped together with few-shot demonstrations [Longpre et al., 2023].

Maintaining VICUNA'S Chatting Ability. VICUNA has demonstrated remarkable chatting ability, achieving 90% of the performance of ChatGPT. This indicates its significant potential as an open-source alternative to closed-source large language models (LLMs) like ChatGPT. To ensure

Dataset Name	Source	Dataset Size
Flan2021	FLAN	388K
Public Pool of Prompts	FLAN	320K
Natural instructions v2	FLAN	200K
CoT	FLAN	100K
Code Search	Husain et al. [2019b]	100K
Code Contest	Li et al. [2022b]	50K
Apps	Hendrycks et al. [2021b]	50K
GPT4-Alpaca	GPT-4	52K
Code-Alpaca	ChatGPT	20K
ShareGPT	ChatGPT	60K
Total	-	1.34M

Table 1: The FLAN-MINI Collection, used to train FLACUNA.

that FLACUNA retains VICUNA's learned knowledge and chatting ability, we incorporated various ChatGPT datasets, including Alpaca [Taori et al., 2023], Code Alpaca [Chaudhary, 2023], and ShareGPT [Chiang et al., 2023], into our FLAN collection. Among these three datasets, VICUNA was originally fine-tuned using the ShareGPT dataset. The final collection was then used to train FLACUNA.

Architecture. We employed LoRA in the VICUNA model for fine-tuning on the FLAN-MINI collection. We inserted the low-rank adapters on all the query and value projection layers, resulting in a total trainable parameter count of 6.55M, which is only around 0.05% of the parameter count of the original 13B VICUNA model. The maximum input sequence length was set to 1280, and efficient training was facilitated by utilizing bf16 precision.

Hyperparameter Details. FLACUNA was trained on 4×A6000 GPUs for 1 epoch. We use 16 gradient accumulation steps with a per-device batch size of 2, resulting in a total batch size of 128. We used 3000 warm-up steps and a learning rate of 2e-5.

3 Evaluation Tasks and Results

3.1 Problem Solving Evaluation

To assess the problem-solving prowess of instructed large language models (LLMs), INSTRUCTEVAL employs a range of benchmarks encompassing real-world exams that delve into diverse topics. These benchmarks encompass complex instructions, arithmetic problems, programming challenges, and causal reasoning tasks. In order to excel in these benchmarks, models need to exhibit a profound understanding of the world, demonstrate multi-hop reasoning capabilities, showcase creativity, and employ a plethora of other cognitive skills.

World Knowledge. The Massive Multitask Language Understanding (MMLU) benchmark, introduced in the work by Hendrycks et al. [2021c], serves as an assessment tool to gauge the problem-solving aptitude and world knowledge of language models across various subjects. It offers evaluations in both zero-shot and few-shot settings, presenting a more challenging and human-like evaluation scenario. The MMLU benchmark encompasses a comprehensive range of 57 subjects spanning STEM, humanities, social sciences, and other domains. The difficulty levels of the tasks within the benchmark vary from elementary to advanced professional levels, providing a comprehensive assessment of the model's capabilities in problem-solving and domain understanding.

Complex Instructions. The subset known as BIG-Bench Hard (BBH) comprises 23 highly demanding tasks carefully selected from the BIG-Bench benchmark [Srivastava et al., 2022] to specifically target tasks that are considered to surpass the current capabilities of language models [Suzgun et al., 2022]. BBH presents models with intricate instructions that require advanced skills in navigation, logical deduction, and fallacy detection.

Comprehension and Arithmetic. Discrete Reasoning Over Paragraphs (DROP) is a reading comprehension task with a mathematical focus. It challenges systems to engage in discrete reasoning by analyzing passages extracted from Wikipedia articles. In order to excel in the DROP task, a system needs to adeptly navigate references within a question and identify the appropriate sections of the provided passage. Additionally, the system must demonstrate proficiency in performing discrete operations like addition, counting, or sorting.

Programming. HumanEval serves as a problem-solving benchmark specifically designed for assessing the performance of large language models that are trained on code [Chen et al., 2021]. The benchmark comprises 164 unique programming problems, encompassing areas such as language comprehension, algorithms, and basic mathematics. Some of the problems included in HumanEval are similar in nature to straightforward software interview questions. In the evaluation process, models are assessed based on the functional correctness of the code programs they generate, with the criteria for correctness determined by the given docstrings. HumanEval provides a comprehensive evaluation framework for assessing the problem-solving capabilities of language models in a code-centric context.

Causality. The Counterfactual Reasoning Assessment (CRASS) benchmark is a novel dataset and evaluation tool developed specifically to assess the causal reasoning abilities of large language models. By employing counterfactual scenarios, CRASS tests the model's capability to identify and select appropriate causal explanations. This benchmark provides a unique and rigorous evaluation framework to gauge the causal reasoning capabilities of language models.

3.2 Alignment to Human Values

Noting the importance of aligning LLMs to human values, INSTRUCTEVAL incorporates the Helpful, Honest, and Harmless (HHH) benchmark [Askell et al., 2021]. The benchmark showcases engaging dialogues between humans and conversational assistants, challenging the model to discern and provide the most appropriate response. It encompasses a diverse array of 61 honesty-related, 59 helpfulness-related, and 58 harmlessness-related samples, along with 43 unique instances falling within the "other" category. The inclusion of the "other" category accounts for examples that embody values not explicitly covered by honesty, helpfulness, or harmlessness.

3.3 Writing Experiments

For the writing experiment, we utilized the IMPACT dataset, which is readily available in INSTRUCTEVAL. This comprehensive dataset consists of 50 prompts across distinct categories, namely informative, professional, argumentative, and creative. Following that, ChatGPT was assigned the responsibility of scoring the models' responses in terms of relevance (Rel.) and coherence (Coh.) on a scale of 1 to 5. For more comprehensive information regarding this evaluation, we refer readers to Chia et al. [2023].

3.4 Results

Comparative Baselines. As baselines, we selected VICUNA [Zheng et al., 2023] and STABLEVICUNA[1].

Few-shot Problem-solving. We present the results of FLACUNA on five datasets (see Table 2) from the INSTRUCTEVAL benchmark, focusing on problem-solving tasks. In 4 out of 5 tasks, FLACUNA outperformed VICUNA, showing an average performance improvement of 5.6 points over the LLaMA backbone. However, it performed slightly worse on code-related problem-solving tasks in the HumanEval dataset, with a margin of 0.6 points. Overall, the improvement in FLACUNA compared to VICUNA is 5.1 points averaged over the five tasks.

Out of the five problem-solving datasets, one of them, DROP, is categorized as a held-in dataset. It is a part of our FLAN collection and was utilized for training FLACUNA. As a result, we observed a significant performance boost of 11 points compared to VICUNA. The remaining datasets are considered held out.

[1] `https://huggingface.co/CarperAI/stable-vicuna-13b-delta`

Model	Size	MMLU (5-shot)		BBH (3-shot)		DROP⋆ (3-shot)		CRASS (3-shot)		HumanEval (0-shot)		Avg.	
		Perf.	Δ	Perf.	Δ	Perf.	Δ	Perf.	Δ	Perf.	Δ	Perf.	Δ
GPT-4	-	86.4	-	-	-	80.9	-	-	-	67.0	-	-	-
ChatGPT	-	70.0	-	49.5	-	64.1	-	90.5	-	48.1	-	64.5	-
Flan-UL2	20B	55.0	-	44.7	-	64.3	-	94.2	-	0.0	-	51.6	-
Alpaca-Lora	30B	58.4	+0.6	41.3	+2.0	45.1	-0.3	79.2	+10.6	18.9	+4.9	48.6	+3.6
OpenAssistant	30B	56.9	-0.9	39.2	-0.1	46.0	+0.6	67.2	+1.4	23.1	+9.1	46.5	+1.5
OPT-IML	30B	38.6	+11.3	31.3	+3.0	47.5	+28.0	67.2	+32.5	9.1	+7.9	38.7	+16.5
Flan-T5	11B	54.5	+29.3	43.9	+13.6	67.2	+49.7	88.3	+54.7	0.0	+0.0	50.8	+29.5
Flan-Alpaca	11B	50.9	+25.7	23.3	-7.0	62.3	+44.8	90.2	+56.6	0.0	+0.0	45.3	+24.0
Dolly V2	12B	25.6	-1.3	29.7	+0.2	16.6	-0.5	35.8	+1.1	8.5	-0.6	23.2	-0.7
Flan-T5	3B	49.2	+25.9	40.2	+15.9	56.3	+43.7	91.2	+60.2	0.0	+0.0	47.4	+29.2
ChatGLM	6B	36.1	-	31.3	-	44.2	-	51.1	-	3.1	-	33.2	-
Mosaic-Chat	7B	37.1	+1.9	32.0	+1.1	20.2	-7.4	47.5	+13.6	17.7	+7.4	30.9	+3.3
STABLEVICUNA	13B	49.2	+3.0	37.5	+0.4	34.3	-1.0	67.5	+8.7	15.9	+2.5	40.9	+2.7
VICUNA	13B	50.6	+4.5	37.6	+0.5	32.6	-3.0	60.9	+2.1	11.6	-1.8	38.7	+0.5
FLACUNA	13B	51.1	+5.0	39.3	+2.2	43.6	+8.0	74.1	+15.3	11.0	-2.4	43.8	+5.6

Table 2: Evaluation results for problem-solving benchmarks. We denote the original performance across the benchmarks as Perf., while Δ denotes the change in performance compared to the corresponding foundation LLMs. ⋆ indicates that DROP is a held-in dataset.

Model	Size	MMLU (0-shot)	BBH (0-shot)	CRASS (0-shot)
Flan-UL2	20B	54.4	34.9	-
OpenAssistant	30B	52.0	33.4	-
OPT IML	30B	41.3	17.4	-
TK-Instruct	11B	39.4	17.1	-
Flan-T5-XXL	11B	54.1	39.5	-
Dolly V2	12B	25.4	22.3	-
STABLEVICUNA	13B	47.5	18.5	64.2
VICUNA	13B	48.3	28.3	65.7
FLACUNA	13B	49.4	32.5	67.9

Table 3: 0-shot problem-solving evaluation of FLACUNA and other baseline models.

0-shot Problem-solving. We conducted a 0-shot performance evaluation of FLACUNA and compared it against both VICUNA and STABLEVICUNA. The results presented in Table 3 demonstrate a noteworthy performance leap by FLACUNA compared to its competitors. This improvement can be attributed to the training of FLACUNA on the high-quality FLAN instruction dataset.

HHH Evaluation. We conducted a further evaluation using BBH's HHH evaluation dataset (see Table 4), where FLACUNA exhibited an impressive 11% improvement over VICUNA. Notably, our instruction dataset collection aimed to enhance VICUNA's problem-solving abilities, but it also had a positive impact on its HHH performance. This observation aligns with the experience of FLAN-T5, which achieved a 24.2% performance improvement over its T5 backbone after fine-tuning on FLAN.

Writing Evaluation. While FLACUNA primarily excels in problem-solving tasks, we made efforts to maintain the impressive writing and chatting ability of VICUNA. To achieve this, we incorporated conversational datasets generated by GPT-4, such as GPT-4-Alpaca and ShareGPT, into the FLAN-MINI collection. However, despite these efforts, we observed certain issues in FLACUNA's writing performance. In some cases, it generates code snippets in response to prompts that are unrelated to coding. We attribute this behavior to the significant data imbalance, where the conversational dataset constitutes only 8.2% of the entire data mixture. Prompt engineering techniques can help rectify such issues.

We discovered that FLACUNA generates responses of reasonable quality when provided with the following template: ‘‘‘`A chat between a curious user and an artificial intelligence assistant. The assistant gives helpful, detailed, and polite answers to the user's questions. USER: definition of the task./n/n question/n Output:`

Model	Size	Harmlessness	Helpfulness	Honesty	Other	Avg.	Δ Avg.
ChatGPT	-	90.7	91.2	78.1	86.3	86.6	-
Flan-Alpaca	11B	74.2	81.4	77.4	83.4	79.1	+26.6
Flan-T5	11B	75.9	75.3	75.1	79.6	76.7	+24.2
Tk-Instruct	11B	70.1	54.8	62.3	76.0	65.8	+13.3
T5	11B	46.4	54.8	58.1	50.7	52.5	-
Alpaca	13B	49.7	51.2	51.8	45.5	49.5	-12.3
LLaMA	13B	57.2	61.0	57.0	72.0	61.8	-
Dolly V2	12B	51.7	59.9	47.0	58.1	54.2	+9.1
Pythia	12B	41.3	46.1	43.6	49.3	45.1	-
StableVicuna	13B	61.7	67.2	57.1	79.1	66.3	+4.5
Vicuna	13B	62.0	66.1	52.4	74.4	63.7	+1.9
Flacuna	13B	72.4	71.2	70.5	83.7	74.5	+12.6

Table 4: Evaluation results for alignment to human values on the honesty, helpfulness, and harmlessness (HHH) benchmark. Avg. denotes the average performance, while Δ Avg. denotes the average improvement compared to the corresponding foundation model.

Model	Size	Informative		Professional		Argumentative		Creative		Avg.	
		Rel.	Coh.	Rel.	Coh.	Rel.	Coh.	Rel.	Coh.	Rel.	Coh.
ChatGPT	-	3.34	3.98	3.88	3.96	3.96	3.82	3.92	3.94	3.78	3.93
Flan-Alpaca	11B	3.56	3.46	3.54	3.70	3.22	3.28	3.70	3.40	3.51	3.46
Flan-T5	11B	2.64	3.24	2.62	3.22	2.54	3.40	2.50	2.72	2.58	3.15
Dolly-V2	12B	3.54	3.64	2.96	3.74	3.66	3.20	3.02	3.18	3.30	3.44
StableVicuna	13B	3.54	3.64	2.96	3.74	3.30	3.20	3.02	3.18	3.21	3.44
Vicuna	13B	3.60	3.96	3.74	3.82	3.82	3.56	3.82	3.92	3.75	3.82
Flacuna	13B	3.02	3.42	3.48	3.52	3.38	3.02	3.92	3.80	3.45	3.44

Table 5: Evaluation results for writing-based tasks.

`ASSISTANT: '''`. This finding suggests that with the appropriate prompts, we can improve Flacuna's chatting performance.

However, upon careful examination of the generated samples, it becomes apparent that Flacuna does not outperform Vicuna as a writing assistant. This observation is reinforced by the evaluation of the generated responses to the prompts in the IMPACT dataset using ChatGPT, as depicted in Table 5. ChatGPT consistently ranks Vicuna's responses significantly higher than those of Flacuna.

The subpar performance of Flacuna in writing-based scenarios can be attributed to several factors. Firstly, the disproportionate scarcity of conversational datasets in Flan may have contributed to this outcome. Additionally, parameter-efficient tuning methods such as LoRA may limit the effectiveness of the model in learning both problem-solving and general writing abilities. Hence, we may explore other efficient training methods for LLMs in the future [Lv et al., 2023].

An example of the prompt and Flacuna's response is shown below.

Prompt: `A chat between a curious user and an artificial intelligence assistant. The assistant gives helpful, detailed, and polite answers to the user's questions. USER: You are tasked to demonstrate your writing skills in professional or work settings for the following question. Write a user manual for a household appliance, providing step-by-step instructions for its operation, maintenance, and troubleshooting, using clear and simple language. Output: ASSISTANT:`

Flacuna's response:

1. Introduction

Welcome to the user manual for the XYZ household appliance. This appliance is designed to make your life easier and more convenient. With this appliance, you can prepare your meals quickly and efficiently.

2. Operation

To operate the appliance, follow these steps:

a. Turn on the appliance by pressing the power button. b. Select the desired cooking mode by using the mode selector. c. Place the food inside the appliance. d. Close the lid and press the start button. e. Wait for the appliance to complete the cooking process.

3. Maintenance

To keep your appliance in good working condition, follow these steps:

a. Clean the appliance after each use. b. Check the appliance for any signs of damage or wear and tear. c. If you notice any issues, contact the manufacturer or a certified repair technician.

4. Troubleshooting

If you encounter any issues with your appliance, try the following troubleshooting steps:

a. Check the power cord and plug to ensure they are securely connected. b. Check the fuse to ensure it is not blown. c. Check the appliance manual for troubleshooting tips. d. If the issue persists, contact the manufacturer or a certified repair technician.

5. Safety Precautions

Always follow these safety precautions when using the appliance:

a. Keep the appliance clean and free of debris. b. Do not use the appliance if it is damaged or showing signs of wear and tear. c. Do not leave the appliance unattended while it is in operation. d. Always use the appliance according to the manufacturer's instructions.

6. Conclusion

Congratulations on your new XYZ household appliance! With proper use and maintenance, your appliance will provide you with years of reliable service. If you have any questions or concerns, please refer to the appliance manual or contact the manufacturer.

---

4 Limitations and Future Work

Despite the promising advancements of FLACUNA compared to VICUNA, we have identified some issues that require addressing:

- If FLACUNA is asked to provide descriptive answers to questions like "Present arguments for or against lowering the age bar for drinking," FLACUNA **generates code snippets instead**. This behavior could be attributed to its **imperfect understanding of instructions or a tendency to hallucinate**.
- FLACUNA is still **significantly behind FLAN-T5** in terms of problem-solving abilities.
- Surprisingly, FLACUNA exhibits **inferior performance compared to both LLAMA and VICUNA on coding-related problems**. This outcome is unexpected, considering that we incorporated numerous coding problem-solving datasets into our instruction tuning collection.
- FLACUNA is **trained with a maximum input sequence length of 1280** which limits its ability to comprehend longer input sequences.

To address these limitations and known issues, we can explore the following steps:

- Based on previous studies, it has been observed that LoRA performs better with larger models [Chia et al., 2023], such as those with 30B or 65B parameters, and excels in task-specific settings. Therefore, in future work, we could enhance FLACUNA by **fully fine-tuning VICUNA, without**

LoRA, particularly on the FLAN collection. Another future work is to train FLACUNA on longer token length.

- We can **incorporate the original FLAN collection into the training process**, as it is fifteen times larger than the instruction dataset we used in this study. FLAN-T5 underwent training on this extensive collection, which resulted in remarkable problem-solving performance.
- The chatting or writing performance of FLACUNA could be improved by **incorporating larger conversational datasets in FLAN-MINI** and subsequently training FLACUNA on it.

References

Yew Ken Chia, Pengfei Hong, Lidong Bing, and Soujanya Poria. Instructeval: Towards holistic evaluation of instruction-tuned large language models, 2023.

Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. Stanford alpaca: An instruction-following llama model, 2023. URL https://github.com/tatsu-lab/stanford_alpaca.

Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. Vicuna: An open-source chatbot impressing gpt-4 with 90%* chatgpt quality, March 2023. URL https://vicuna.lmsys.org.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aur'elien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. Llama: Open and efficient foundation language models. *ArXiv*, abs/2302.13971, 2023.

Shayne Longpre, Le Hou, Tu Vu, Albert Webson, Hyung Won Chung, Yi Tay, Denny Zhou, Quoc V Le, Barret Zoph, Jason Wei, et al. The flan collection: Designing data and methods for effective instruction tuning. *arXiv preprint arXiv:2301.13688*, 2023.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Ré mi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel J. Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with AlphaCode. *Science*, 378(6624):1092–1097, dec 2022a. doi: 10.1126/science.abq1158. URL https://doi.org/10.1126%2Fscience.abq1158.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Xiaodong Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. *ArXiv*, abs/2105.09938, 2021a.

Hamel Husain, Hongqi Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. *ArXiv*, abs/1909.09436, 2019a.

Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. CodeSearchNet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019b.

Yujia Li, David Choi, Junyoung Chung, Nate Kushman, Julian Schrittwieser, Rémi Leblond, Tom Eccles, James Keeling, Felix Gimeno, Agustin Dal Lago, Thomas Hubert, Peter Choy, Cyprien de Masson d'Autume, Igor Babuschkin, Xinyun Chen, Po-Sen Huang, Johannes Welbl, Sven Gowal, Alexey Cherepanov, James Molloy, Daniel Mankowitz, Esme Sutherland Robson, Pushmeet Kohli, Nando de Freitas, Koray Kavukcuoglu, and Oriol Vinyals. Competition-level code generation with alphacode. *arXiv preprint arXiv:2203.07814*, 2022b.

Dan Hendrycks, Steven Basart, Saurav Kadavath, Mantas Mazeika, Akul Arora, Ethan Guo, Collin Burns, Samir Puranik, Horace He, Dawn Song, and Jacob Steinhardt. Measuring coding challenge competence with apps. *NeurIPS*, 2021b.

Sahil Chaudhary. Code alpaca: An instruction-following llama model for code generation. https://github.com/sahil280114/codealpaca, 2023.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021c. URL https://openreview.net/forum?id=d7KBjmI3GmQ.

Aarohi Srivastava, Abhinav Rastogi, Abhishek Rao, Abu Awal Md Shoeb, Abubakar Abid, Adam Fisch, Adam R. Brown, Adam Santoro, Aditya Gupta, Adrià Garriga-Alonso, Agnieszka Kluska, Aitor Lewkowycz, Akshat Agarwal, Alethea Power, Alex Ray, Alex Warstadt, Alexander W. Kocurek, Ali Safaya, Ali Tazarv, Alice Xiang, Alicia Parrish, Allen Nie, Aman Hussain, Amanda Askell, Amanda Dsouza, Ambrose Slone, Ameet Rahane, Anantharaman S. Iyer, Anders Andreassen, Andrea Madotto, Andrea Santilli, Andreas Stuhlmüller, Andrew Dai, Andrew La, Andrew Lampinen, Andy Zou, Angela Jiang, Angelica Chen, Anh Vuong, Animesh Gupta, Anna Gottardi, Antonio Norelli, Anu Venkatesh, Arash Gholamidavoodi, Arfa Tabassum, Arul Menezes, Arun Kirubarajan, Asher Mullokandov, Ashish Sabharwal, Austin Herrick, Avia Efrat, Aykut Erdem, Ayla Karakaş, B. Ryan Roberts, Bao Sheng Loe, Barret Zoph, Bartłomiej Bojanowski, Batuhan Özyurt, Behnam Hedayatnia, Behnam Neyshabur, Benjamin Inden, Benno Stein, Berk Ekmekci, Bill Yuchen Lin, Blake Howald, Cameron Diao, Cameron Dour, Catherine Stinson, Cedrick Argueta, César Ferri Ramírez, Chandan Singh, Charles Rathkopf, Chenlin Meng, Chitta Baral, Chiyu Wu, Chris Callison-Burch, Chris Waites, Christian Voigt, Christopher D. Manning, Christopher Potts, Cindy Ramirez, Clara E. Rivera, Clemencia Siro, Colin Raffel, Courtney Ashcraft, Cristina Garbacea, Damien Sileo, Dan Garrette, Dan Hendrycks, Dan Kilman, Dan Roth, Daniel Freeman, Daniel Khashabi, Daniel Levy, Daniel Moseguí González, Danielle Perszyk, Danny Hernandez, Danqi Chen, Daphne Ippolito, Dar Gilboa, David Dohan, David Drakard, David Jurgens, Debajyoti Datta, Deep Ganguli, Denis Emelin, Denis Kleyko, Deniz Yuret, Derek Chen, Derek Tam, Dieuwke Hupkes, Diganta Misra, Dilyar Buzan, Dimitri Coelho Mollo, Diyi Yang, Dong-Ho Lee, Ekaterina Shutova, Ekin Dogus Cubuk, Elad Segal, Eleanor Hagerman, Elizabeth Barnes, Elizabeth Donoway, Ellie Pavlick, Emanuele Rodola, Emma Lam, Eric Chu, Eric Tang, Erkut Erdem, Ernie Chang, Ethan A. Chi, Ethan Dyer, Ethan Jerzak, Ethan Kim, Eunice Engefu Manyasi, Evgenii Zheltonozhskii, Fanyue Xia, Fatemeh Siar, Fernando Martínez-Plumed, Francesca Happé, Francois Chollet, Frieda Rong, Gaurav Mishra, Genta Indra Winata, Gerard de Melo, Germán Kruszewski, Giambattista Parascandolo, Giorgio Mariani, Gloria Wang, Gonzalo Jaimovitch-López, Gregor Betz, Guy Gur-Ari, Hana Galijasevic, Hannah Kim, Hannah Rashkin, Hannaneh Hajishirzi, Harsh Mehta, Hayden Bogar, Henry Shevlin, Hinrich Schütze, Hiromu Yakura, Hongming Zhang, Hugh Mee Wong, Ian Ng, Isaac Noble, Jaap Jumelet, Jack Geissinger, Jackson Kernion, Jacob Hilton, Jaehoon Lee, Jaime Fernández Fisac, James B. Simon, James Koppel, James Zheng, James Zou, Jan Kocoń, Jana Thompson, Jared Kaplan, Jarema Radom, Jascha Sohl-Dickstein, Jason Phang, Jason Wei, Jason Yosinski, Jekaterina Novikova, Jelle Bosscher, Jennifer Marsh, Jeremy Kim, Jeroen Taal, Jesse Engel, Jesujoba Alabi, Jiacheng Xu, Jiaming Song, Jillian Tang, Joan Waweru, John Burden, John Miller, John U. Balis, Jonathan Berant, Jörg Frohberg, Jos Rozen, Jose Hernandez-Orallo, Joseph Boudeman, Joseph Jones, Joshua B. Tenenbaum, Joshua S. Rule, Joyce Chua, Kamil Kanclerz, Karen Livescu, Karl Krauth, Karthik Gopalakrishnan, Katerina Ignatyeva, Katja Markert, Kaustubh D. Dhole, Kevin Gimpel, Kevin Omondi, Kory Mathewson, Kristen Chiafullo, Ksenia Shkaruta, Kumar Shridhar, Kyle McDonell, Kyle Richardson, Laria Reynolds, Leo Gao, Li Zhang, Liam Dugan, Lianhui Qin, Lidia Contreras-Ochando, Louis-Philippe Morency, Luca Moschella, Lucas Lam, Lucy Noble, Ludwig Schmidt, Luheng He, Luis Oliveros Colón, Luke Metz, Lütfi Kerem Şenel, Maarten Bosma, Maarten Sap, Maartje ter Hoeve, Maheen Farooqi, Manaal Faruqui, Mantas Mazeika, Marco Baturan, Marco Marelli, Marco Maru, Maria Jose Ramírez Quintana, Marie Tolkiehn, Mario Giulianelli, Martha Lewis, Martin Potthast, Matthew L. Leavitt, Matthias Hagen, Mátyás Schubert, Medina Orduna Baitemirova, Melody Arnaud, Melvin McElrath, Michael A. Yee, Michael Cohen, Michael Gu, Michael Ivanitskiy, Michael Starritt, Michael Strube, Michał Swędrowski, Michele Bevilacqua, Michihiro Yasunaga, Mihir Kale, Mike Cain, Mimee Xu, Mirac Suzgun, Mo Tiwari, Mohit Bansal, Moin Aminnaseri, Mor Geva, Mozhdeh Gheini, Mukund Varma T, Nanyun Peng, Nathan Chi, Nayeon Lee, Neta Gur-Ari Krakover, Nicholas Cameron, Nicholas Roberts, Nick Doiron, Nikita

Nangia, Niklas Deckers, Niklas Muennighoff, Nitish Shirish Keskar, Niveditha S. Iyer, Noah Constant, Noah Fiedel, Nuan Wen, Oliver Zhang, Omar Agha, Omar Elbaghdadi, Omer Levy, Owain Evans, Pablo Antonio Moreno Casares, Parth Doshi, Pascale Fung, Paul Pu Liang, Paul Vicol, Pegah Alipoormolabashi, Peiyuan Liao, Percy Liang, Peter Chang, Peter Eckersley, Phu Mon Htut, Pinyu Hwang, Piotr Miłkowski, Piyush Patil, Pouya Pezeshkpour, Priti Oli, Qiaozhu Mei, Qing Lyu, Qinlang Chen, Rabin Banjade, Rachel Etta Rudolph, Raefer Gabriel, Rahel Habacker, Ramón Risco Delgado, Raphaël Millière, Rhythm Garg, Richard Barnes, Rif A. Saurous, Riku Arakawa, Robbe Raymaekers, Robert Frank, Rohan Sikand, Roman Novak, Roman Sitelew, Ronan LeBras, Rosanne Liu, Rowan Jacobs, Rui Zhang, Ruslan Salakhutdinov, Ryan Chi, Ryan Lee, Ryan Stovall, Ryan Teehan, Rylan Yang, Sahib Singh, Saif M. Mohammad, Sajant Anand, Sam Dillavou, Sam Shleifer, Sam Wiseman, Samuel Gruetter, Samuel R. Bowman, Samuel S. Schoenholz, Sanghyun Han, Sanjeev Kwatra, Sarah A. Rous, Sarik Ghazarian, Sayan Ghosh, Sean Casey, Sebastian Bischoff, Sebastian Gehrmann, Sebastian Schuster, Sepideh Sadeghi, Shadi Hamdan, Sharon Zhou, Shashank Srivastava, Sherry Shi, Shikhar Singh, Shima Asaadi, Shixiang Shane Gu, Shubh Pachchigar, Shubham Toshniwal, Shyam Upadhyay, Shyamolima, Debnath, Siamak Shakeri, Simon Thormeyer, Simone Melzi, Siva Reddy, Sneha Priscilla Makini, Soo-Hwan Lee, Spencer Torene, Sriharsha Hatwar, Stanislas Dehaene, Stefan Divic, Stefano Ermon, Stella Biderman, Stephanie Lin, Stephen Prasad, Steven T. Piantadosi, Stuart M. Shieber, Summer Misherghi, Svetlana Kiritchenko, Swaroop Mishra, Tal Linzen, Tal Schuster, Tao Li, Tao Yu, Tariq Ali, Tatsu Hashimoto, Te-Lin Wu, Théo Desbordes, Theodore Rothschild, Thomas Phan, Tianle Wang, Tiberius Nkinyili, Timo Schick, Timofei Kornev, Timothy Telleen-Lawton, Titus Tunduny, Tobias Gerstenberg, Trenton Chang, Trishala Neeraj, Tushar Khot, Tyler Shultz, Uri Shaham, Vedant Misra, Vera Demberg, Victoria Nyamai, Vikas Raunak, Vinay Ramasesh, Vinay Uday Prabhu, Vishakh Padmakumar, Vivek Srikumar, William Fedus, William Saunders, William Zhang, Wout Vossen, Xiang Ren, Xiaoyu Tong, Xinran Zhao, Xinyi Wu, Xudong Shen, Yadollah Yaghoobzadeh, Yair Lakretz, Yangqiu Song, Yasaman Bahri, Yejin Choi, Yichi Yang, Yiding Hao, Yifu Chen, Yonatan Belinkov, Yu Hou, Yufang Hou, Yuntao Bai, Zachary Seid, Zhuoye Zhao, Zijian Wang, Zijie J. Wang, Zirui Wang, and Ziyi Wu. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models, 2022.

Mirac Suzgun, Nathan Scales, Nathanael Scharli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed Huai hsin Chi, Denny Zhou, and Jason Wei. Challenging big-bench tasks and whether chain-of-thought can solve them. *ArXiv*, abs/2210.09261, 2022.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde, Jared Kaplan, Harrison Edwards, Yura Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, David W. Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William H. Guss, Alex Nichol, Igor Babuschkin, S. Arun Balaji, Shantanu Jain, Andrew Carr, Jan Leike, Joshua Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew M. Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. *ArXiv*, abs/2107.03374, 2021.

Amanda Askell, Yuntao Bai, Anna Chen, Dawn Drain, Deep Ganguli, Tom Henighan, Andy Jones, Nicholas Joseph, Ben Mann, Nova DasSarma, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Jackson Kernion, Kamal Ndousse, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, and Jared Kaplan. A general language assistant as a laboratory for alignment, 2021.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric. P Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging llm-as-a-judge with mt-bench and chatbot arena, 2023.

Kai Lv, Yuqing Yang, Tengxiao Liu, Qinghui Gao, Qipeng Guo, and Xipeng Qiu. Full parameter fine-tuning for large language models with limited resources, 2023.